

Obicall Architecture and Implementation Strategy

Introduction and objectives

Obicall is a proposed dynamic polyglot runtime for sensor fusion, implemented in C with CMake, a command line interface, and platform-specific shared libraries. Its central design is two isolated runtime brokers that coordinate interchangeable language providers and maintain a recoverable estimation pipeline. Dimensional game theory supplies an experimental policy for selecting processing strategies across uncertainty, timeliness, availability, and resource cost.

The initial objective is a reproducible desktop prototype that survives specified worker or broker failures, preserves measurement provenance, and explicitly reports when an estimate becomes unusable. Continued execution and correct estimation are separate requirements. This architecture does not establish hard real-time guarantees, host-failure tolerance, or automotive safety qualification.

Reference basis

The design draws architectural ideas from four reference projects, without reproducing their implementation code.¹

Reference	Useful foundation	Obicall design implication
LibPolyCall	Opaque C contexts and separation of bindings from application logic	Keep estimation outside language adapters
NSIGII dual FFI	Native calls with an alternative message transport	Add process isolation and explicit failover authority
Dynamic C ABI Loader	Manifest discovery, hashes, and provider cost selection	Resolve every required dependency before ranking alternatives
OBIAI Tensor TTS v0.2	Explicit tensor metadata and replayable diagnostic state	Define sensor-specific uncertainty and replay schemas

These are design references, not validated Obicall components. For example, the NSIGII Python bridge selects fallback when library loading fails; that does not establish recovery from a crashing native call. The loader follows one shortest path, which can omit another mandatory dependency. Obicall must distinguish alternative providers from dependencies that are all required.

Architecture and broker responsibilities

“Dual FFI” means two independently brokered endpoints using the same bidirectional C contract. Each supports application requests and provider events. It does not limit Obicall to two programming languages. Foreign calls occur within a process; communication between processes uses serialized messages.

¹ OBINexus reference archives; exact files and sections appear in Sources 1.

Sensor adapters publish validated observations into a bounded admission journal. Both brokers consume identical window identifiers and admission decisions. Broker A initially publishes results; Broker B runs as a hot shadow, maintaining its own estimator state. Each owns separate provider worker processes, so a Python runtime or native plugin crash cannot directly corrupt the other broker's address space.

A supervisor restarts failed processes with bounded retry rates. A separate downstream output gate controls publication. This gate is an authority component, not a third runtime broker. Its failure stops valid publication; consumers must expire old results and execute their application-defined safe response. The initial deployment assumes a functioning operating system and output gate. A failed admission journal also stops affected pipelines; shared input infrastructure requires separate redundancy.

Foreign function contract

Define a versioned entry point such as `obicall_plugin_query_v1`, returning a function table for creation, submission, polling, checkpointing, restoration, and destruction. Include ABI version, structure size, capability identifiers, and supported schema versions. Use fixed-width numeric fields, opaque handles, and pointer-plus-length buffers. Specify calling convention and target architecture explicitly.

Inputs remain borrowed only for the documented call duration. Asynchronous submissions copy data or retain it through an explicit ownership API. The allocating module provides the matching release operation. Never exchange C++ objects, runtime-managed objects, or process-local pointers over IPC. Encode messages field by field with fixed byte order and bounded lengths; native structure padding is not a wire format.

Start with fixed function tables. Optional libffi support can later enable declared dynamic signatures; libffi supplies calling-convention machinery, while Obicall must still implement type conversion and ownership rules.² Finding an exported symbol does not prove its declared signature is correct. Require descriptor negotiation and adapter conformance tests.

Dynamic loading and language integration

A provider manifest declares capabilities, mandatory dependencies, ABI, architecture, artifact digests, and configuration schema. Resolve the complete dependency graph in topological order, rejecting cycles. Rank eligible alternative providers only after compatibility and integrity checks pass. Trusted manifests and protected artifact directories are required; a hash alone does not authenticate a publisher.

Load approved bundles inside workers using `dlopen` and `dlsym` on Linux/macOS, or `LoadLibraryExW` and `GetProcAddress` on Windows. Restrict dependency search locations. Library initialization can execute during loading, before a plugin query occurs.³ Replace workers to upgrade modules: start, validate, restore, switch routing, drain, then terminate the old worker. Avoid unloading code while calls or callbacks remain active.

² libffi project, [Foreign Function Interface overview](#).

³ Platform documentation: [Linux dlopen](#), [Windows LoadLibraryExW](#), and [Apple dynamic libraries](#).

Implement C and Python providers first; add a Node-API adapter subsequently. Python callback objects need retained references, and Node callbacks require dispatch through the appropriate event-loop mechanisms.⁴ Adapters translate values and errors; they do not implement fusion policy. A provider is eligible for failover only if its capabilities, schemas, and checkpoint format match the required pipeline.

Fault tolerance and publication safety

Heartbeats report liveness and processing progress, including the last completed window. A timeout marks a broker suspect; it does not independently grant the peer publishing rights.

The output gate issues a monotonically increasing ownership epoch. Every result carries broker identity, epoch, pipeline/window identifier, input digest, configuration digest, and validity deadline. The gate accepts only its current owner and rejects expired or duplicate windows. Downstream enforcement of ownership generations follows the fencing principle illustrated by Chubby sequencers.⁵

On active failure, the gate revokes the old epoch. Promotion requires a healthy shadow with compatible state and sufficient replay progress. Only then does the gate grant a new epoch. Delayed results from the old broker remain invalid. Checkpoints contain estimator state, covariance, source boot identifiers and sequence positions, pending windows, and calibration/model/policy versions. Recovery never serializes native pointers.

Persist ownership and committed-window state before acknowledging publication. If recovery cannot establish that state, keep output invalid until reconciliation. Retry read-only computation by window identifier; do not describe this as exactly-once execution of external effects.

Two disagreeing replicas cannot identify the correct estimate merely by comparing themselves. Compare equal-input results within declared numeric tolerances; unexplained disagreement triggers degradation or invalid output. Identical brokers can also share software or sensor faults. For a future multi-host deployment, use an independently reliable authority; a two-voter Raft group loses its majority when one voter is unavailable.⁶

Sensor stream integration and estimation

Each observation carries sensor identity, source sequence, sampling time, arrival time, clock domain and timing uncertainty, coordinate frame, units, calibration version, payload shape, and measurement covariance. Validate finite values, dimensions, lengths, and covariance suitability before admission.

Transform observations into declared reference frames and units. Use measurement models for different modalities rather than averaging incompatible quantities. Maintain bounded reordering windows and explicit queue-overflow policies. Record rejected and missing observations; never substitute zero for absent data. Propagate state to the estimation time and enforce an application-specific maximum age.

⁴ [Python ctypes callbacks](#) and [Node-API thread-safe calls](#).

⁵ Burrows, [The Chubby lock service](#), 2006, section 2.4.

⁶ Ongaro and Ousterhout, [Raft extended paper](#), 2014, section 5.2.

Begin with a linear Kalman estimator on simulated position measurements. Its probabilistic interpretation depends on the stated linear/Gaussian model and noise assumptions.⁷ As an illustrative scalar check, independent unbiased readings of 10.0 m and 10.4 m, with variances 0.04 m² and 0.16 m², produce an inverse-variance fused estimate of 10.08 m and variance 0.032 m². These are calculated example values, not benchmark results.

Track shared information to avoid counting it twice. Unknown cross-correlations between tracks may justify covariance intersection, subject to appropriate input covariance bounds; it does not repair arbitrary sensor bias.⁸ The two brokers' outputs are replicas of the same evidence and must not be fused as independent measurements. Every published estimate includes uncertainty, contributing sources, timestamp, and status: valid, degraded, or invalid.

Dimensional game theory policy

Nnamdi Michael Okpala's DGT manuscript proposes multidimensional strategies and adaptive weighting.⁹ For Obicall, treat this as a research policy with measurable hypotheses. Its unrestricted claim that perfect zero-sum play always yields a tie is not a valid general premise: even a game with constant payoffs of +1 and -1 is zero-sum. Nash equilibrium describes resistance to unilateral improvement, not necessarily a tie.¹⁰

Define the controller's actions as approved combinations of sensor subsets, estimator modes, and providers. Define the opposing side as a finite catalog of fault scenarios, such as dropout, bias, or processing delay. Sensors themselves are not assumed to be strategic agents.

Assign each action/scenario pair a dimensionless loss using documented normalization scales and nonnegative weights for estimation error, delay, loss of coverage, and resource consumption. Estimate these losses from simulation and recorded data; do not imply that ground-truth error is directly observable online.

First eliminate actions violating freshness, capability, observability, or ownership constraints. Among the remaining actions, select the one with the smallest worst-case scenario loss, using deterministic tie-breaking. This is a restricted minimax policy, not a general equilibrium solver. Apply switching hysteresis; on policy timeout or invalid scores, use a previously validated eligible baseline or report invalid output. DGT must never override publication safeguards or invent covariance values.

For a synthetic policy test, suppose camera-heavy, inertial-heavy, and balanced actions have worst-case losses of 0.85, 0.70, and 0.35. Select the balanced action if eligible. These illustrative values must be replaced by measured, versioned scenario costs.

Implementation sequence and build strategy

1. **Freeze contracts.** Specify ABI, observation/result schemas, ownership, error codes, and replay format. Create a C reference provider and deterministic sensor recordings.
2. **Build one complete path.** Connect C and Python sensor producers through one broker to the estimator and output gate. Verify the numerical example and malformed-input rejection.
3. **Add the second broker.** Implement journal replay, checkpoints, epoch fencing, readiness checks, and worker replacement. Demonstrate broker death followed by controlled promotion.

⁷ Särkkä, [Bayesian Filtering and Smoothing](#), 2013, chapter 4.

⁸ Cros et al., [Optimality of Split Covariance Intersection Fusion](#), 2023, sections 1–2.

⁹ Okpala, [Dimensional Game Theory](#), 22 May 2025, sections 3–4; supplied ReADME(7).md.

¹⁰ Nash, [Non-Cooperative Games](#), 1951.

4. **Introduce DGT.** Add a versioned scenario/loss table and compare its decisions against a fixed-policy baseline before enabling adaptation.

5. **Package the platform builds.** Create `obicall_core` as a CMake SHARED target and separate CLI, broker, and worker executable targets. Use generated export macros, hidden internal symbols, and `${CMAKE_DL_LIBS}` where applicable.¹¹

Set the core output name to `obicall`; clear the Windows library prefix explicitly. Expected artifacts are `libobicall.so` on Linux, `obicall.dll` plus the toolchain import library on Windows, and `libobicall.dylib` on macOS. Install public headers and CMake package metadata. Build each platform separately; renaming a binary does not change its ABI.

Use the MSYS2 UCRT64 shell and matching native Windows packages, rather than mixing toolchains. MSYS2 documents UCRT64 and explicit Ninja selection.¹² Illustrative build commands, once the proposed source targets exist, are:

```
pacman -S --needed mingw-w64-ucrt-x86_64-gcc \
  mingw-w64-ucrt-x86_64-cmake mingw-w64-ucrt-x86_64-ninja
cmake -S . -B build/ucrt64 -G Ninja -DCMAKE_BUILD_TYPE=Debug
cmake --build build/ucrt64
ctest --test-dir build/ucrt64 --output-on-failure
```

The proposed CLI exposes `doctor`, `validate`, `run`, `status`, and `replay`, with machine-readable JSON diagnostics and documented exit codes. `doctor` checks architecture, ABI, provider availability, and configuration. Fault injection belongs in test builds. These commands describe the implementation target; no completed Obicall runtime is asserted here.

Testing and validation

Use CTest to orchestrate ABI, serialization, numerical, and process-level tests across Linux, Windows UCRT64, and macOS. Exercise incompatible ABI versions, missing transitive dependencies, allocator mismatches, callback shutdown, oversized payloads, clock resets, reordered packets, sensor bias, worker hangs, broker crashes, and gate restarts. Use sanitizers and parser fuzzing where supported.

Acceptance requires rejection of stale epochs and duplicate commits, bounded queues, explicit invalid output without adequate observations, and reproducible replay within documented numerical tolerances. Inject a delayed old-owner result after promotion: it must be rejected. Measure detection, recovery, end-to-end latency percentiles, memory, dropped samples, estimation error, and uncertainty calibration. Fix thresholds against application requirements before release; report actual hardware and workloads.

Evaluate DGT on held-out fault scenarios against fixed-policy fusion. Record policy-switch frequency and computational overhead alongside accuracy. Improved loss on a training scenario table is insufficient evidence of general reliability.

Conclusion

Obicall can provide a practical foundation for polyglot sensor fusion by combining explicit C contracts, isolated providers, replicated estimation, and controlled publication. Its research contribution would be a measurable DGT policy operating within these constraints. The decisive milestone is a reproducible failure-and-recovery demonstration with trustworthy uncertainty reporting, followed by comparative evaluation of adaptive sensor selection.

¹¹ Kitware: [add_library](#), [GenerateExportHeader](#), [CMAKE_DL_LIBS](#).

¹² MSYS2, [Environments](#) and [Using CMake](#).

Sources

1. OBINexus. Supplied archives: libpolycall-main (1).zip, include/polycall.h and docs/ARCHITECTURE.md; nsigii-p2p-dual-ffi-main.zip, node.py and main.go; dynamic-cabi-loader-main.zip, python/dynloader/runtime.py; obiai_tensor_tts-main.zip, nested v0.2.0 README.md, docs/MATH.md and bayesian_state.py. Private attachments, inspected 14 September 2026.
2. libffi project. [Foreign Function Interface overview](#). Official documentation, accessed 14 September 2026.
3. Linux man-pages, [dlopen](#); Microsoft, [LoadLibraryExW](#); Apple, [Dynamic Library Usage Guidelines](#) (archived; cited for loading semantics). Accessed 14 September 2026.
4. Python Software Foundation, [ctypes callback functions](#); Node.js, [Node-API asynchronous thread-safe calls](#). Official documentation, accessed 14 September 2026.
5. Burrows, M. [The Chubby lock service for loosely-coupled distributed systems](#). OSDI, 2006, section 2.4.
6. Ongaro, D., and Ousterhout, J. [In Search of an Understandable Consensus Algorithm](#). Extended version, 2014, section 5.2.
7. Särkkä, S. [Bayesian Filtering and Smoothing](#). Cambridge University Press, 2013, chapter 4. First-edition mathematical reference.
8. Cros, C., Amblard, P.-O., Prieur, C., and Da Rocha, J.-F. [Optimality of Split Covariance Intersection Fusion](#). arXiv:2307.14741v1, 2023, sections 1–2; research preprint.
9. Okpala, N. M. Dimensional Game Theory: A Framework for Strategic Algorithm Development. OBINexus Computing, 22 May 2025, sections 3–4. Supplied ReADME(7).md; research proposal.
10. Nash, J. [Non-Cooperative Games](#). Annals of Mathematics, 54(2), 286–295, 1951.
11. Kitware. CMake documentation: [add_library](#), [GenerateExportHeader](#), and [CMAKE_DL_LIBS](#). Accessed 14 September 2026.
12. MSYS2. [Environments](#) and [Using CMake in MSYS2](#). Official documentation, accessed 14 September 2026.